

5장 합성곱 신경망1 (~5.2)

5.1 합성곱 신경망

Convolutional Neural Network = ConvNet = CNN

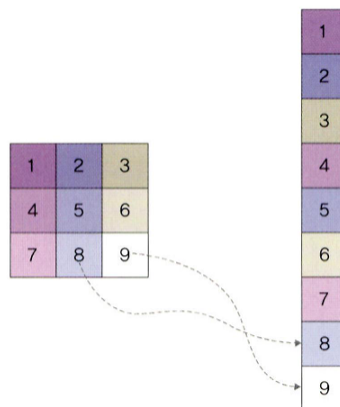
역전파에서는 순전파 과정에 따라 계산된 오차 정보가 신경망의 모든 노드로 전송됨

이런 과정은 복잡하고 많은 자원을 요구함

⇒ 합성곱 신경망 : 이미지 전체가 아닌 일부만 계산함으로써 시간과 자원을 절약하여 이미지의 세밀한 부분까지 분석할 수 있는 신경망

합성곱층의 필요성

주로 음성 인식이나 이미지, 영상 처리에 유용한 신경망



이미지 분석 시 3x3 배열을 펼쳐서 각 픽셀에 가중치를 곱하여 은닉층으로 전달함

이미지를 펼치게 되면 데이터의 공간적 구조를 무시하게 됨

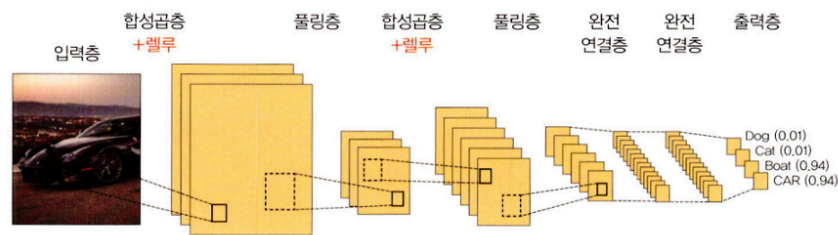
이를 방지하기 위해 합성곱층을 도입

합성곱 신경망 구조 (CNN)

다차원 배열 데이터를 처리하도록 구성되어 컬러 이미지 같은 다차원 배열 처리에 특화되어 있음

5개의 계층으로 구성됨

▼ 그림 5-2 합성곱 신경망 구조



합성곱층과 풀링층을 거치면서 입력 이미지의 주요 특성 벡터를 추출
추출된 주요 특성 벡터들은 완전 연결층을 거치면서 1차원 벡터로 변환됨
마지막으로 출력층에서 활성화 함수인 softmax 함수를 사용하여 최종 결과를 출력

• 입력층

입력 이미지 데이터가 최초로 거치게 되는 계층
이미지는 높이, 너비, 채널의 값을 갖는 3차원 데이터
채널은 이미지가 gray scale이면 1, 컬러(RGB)이면 3 값을 가짐
예를 들어 높이 4, 너비 4, RGB를 가진 채널의 이미지 형태는 (4, 4, 3)으로 표현 가능

• 합성곱층

입력 데이터에서 특성을 추출하는 역할을 수행
입력 이미지가 들어오면 이미지에 대한 특성을 감지하기 위해 kernel이나 필터 사용
커널/필터는 이미지의 모든 영역을 훑으면서 특성을 추출
추출된 결과물이 특성 맵 (feature map)
일반적으로 커널은 3x3, 5x5 크기로 사용
스트라이드(stride)라는 지정된 간격에 따라 순차적으로 이동

이미지 크기가 (6, 6, 1)이며 3x3 크기의 커널/필터가 스트라이드 1 간격으로 이동하면서 합성곱 연산을 수행한 예시



$$(0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (0 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 0) + (1 \times 1) = 1$$

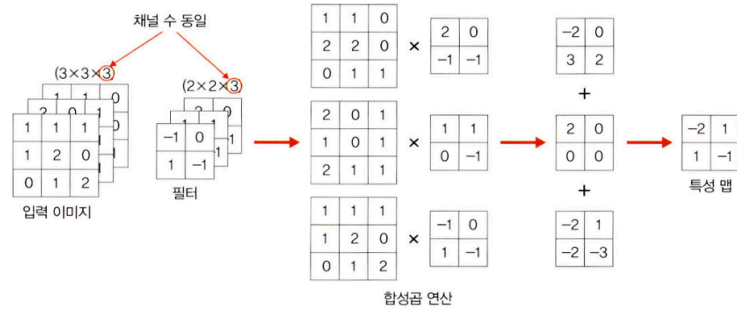
▼ 그림 5-5 입력 이미지에 필터가 1만큼 이동



커널은 스트라이드 간격만큼 순회하면서 모든 입력 값과 합성곱 연산으로 새로운 특성 맵을 만들게 됨 → (4, 4, 1) 크기의 특성 맵으로 줄어들음

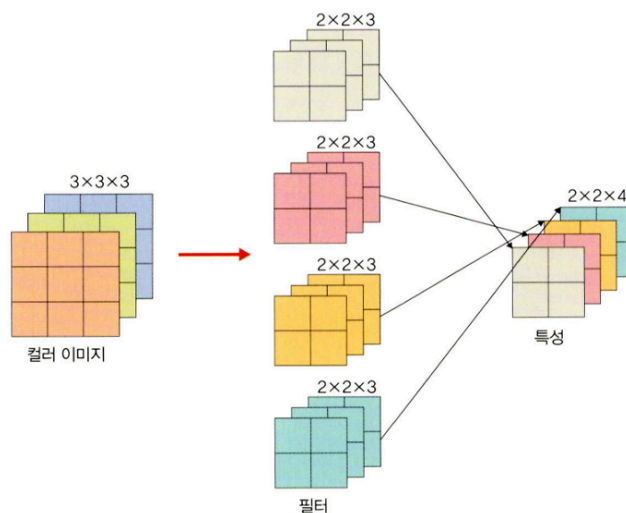
컬러 이미지의 합성곱 예시

♥ 그림 5-10 컬러 이미지 합성곱



RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후 결과를 더해줌
필터 채널이 3이지만 필터 개수는 3개가 아닌 1개

필터가 2개 이상인 합성곱 예시



[합성곱층 요약]

- 입력 데이터 : $W_1 * H_1 * D_1$ (W_1 : 가로, H_1 : 세로, D_1 : 채널/깊이)
- 하이퍼파라미터
 - 필터 개수 : K
 - 필터 크기 : F
 - 스트라이드 : S
 - 패딩 : P

- 출력 데이터

$$W_2 = (W_1 - F + 2P)/S + 1$$

$$H_2 = (H_1 - F + 2P)/S + 1$$

$$D_2 = K$$

- 풀링층

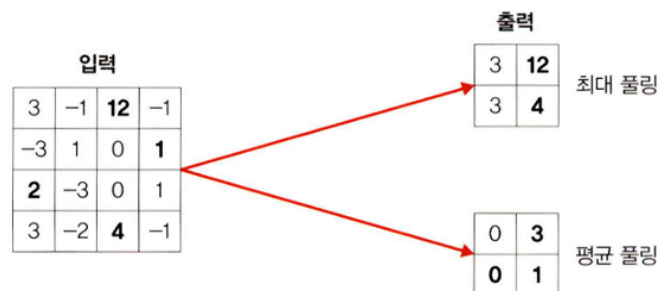
합성곱층과 유사하게 특성 맵의 차원을 다운 샘플링(이미지 축소)하여 연산량을 감소시키고 주요한 특성 벡터를 추출하여 학습을 효과적으로 할 수 있게 함

풀링 연산

- Max Pooling (최대 풀링) : 대상 영역에서 최댓값을 추출
- Average Pooling (평균 풀링) : 대상 영역에서 평균을 반환

대부분의 합성곱 신경망에서는 최대 풀링을 사용

평균 풀링은 각 커널 값을 평균화 시켜 중요한 가중치를 갖는 값의 특성이 희미해질 수 있음



평균 풀링은 각 필터의 평균을 계산

[풀링층 요약]

- 입력 데이터 : $W_1 * H_1 * D_1$
- 하이퍼파라미터
필터 크기 : F
스트라이드 : S
- 출력 데이터
 $W_2 = (W_1 - F) / S + 1$
 $H_2 = (H_1 - F) / S + 1$
 $D_2 = D_1$

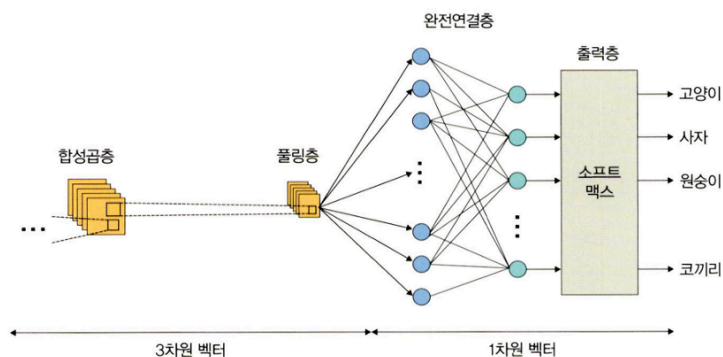
▼ 합성곱층과 풀링층의 차이

합성곱층 : 이미지에서 어떤 특징이 있는지 찾아내는 역할
필터의 값 (가중치)을 학습

풀링층 : 합성곱층에서 만들어진 특징 맵의 크기를 줄이는 역할
정보를 어느 정도 유지하면서 중요한 값만 남기는 압축 과정

풀링층은 합성곱층처럼 새로운 특징을 학습해서 추출하지 않음
특징을 선택, 강조하는 효과를 가짐 (선택하고 요약)

• 완전 연결층



합성곱층과 풀링층을 거치면서 차원이 축소된 특성 맵은 최종적으로 완전 연결층으로 전달 됨

이 과정에서 이미지는 3차원 벡터에서 1차원 벡터로 펼쳐지게 됨

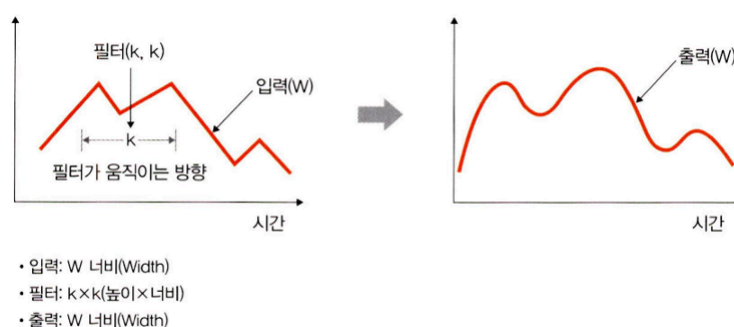
• 출력층

소프트맥스 활성화 함수를 사용하여 입력 받은 값을 0~1 사이의 값으로 출력
이미지가 각 레이블에 속할 확률 값을 출력
가장 높은 확률 값을 갖는 레이블이 최종 값으로 선정됨

1D, 2D, 3D 합성곱

합성곱은 이동하는 방향의 수와 출력의 형태에 따라 분류

- 1D 합성곱



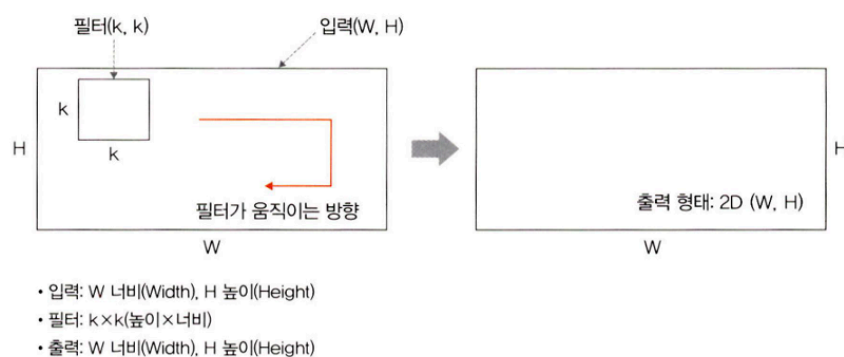
필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱

입력 W와 필터 k에 대한 출력은 W가 됨

그래프 곡선을 완화할 때 많이 사용됨

예시) 입력이 [1, 1, 1, 1, 1]이고 필터가 [0.25, 0.5, 0.25] → 출력은 [1, 1, 1]

- 2D 합성곱

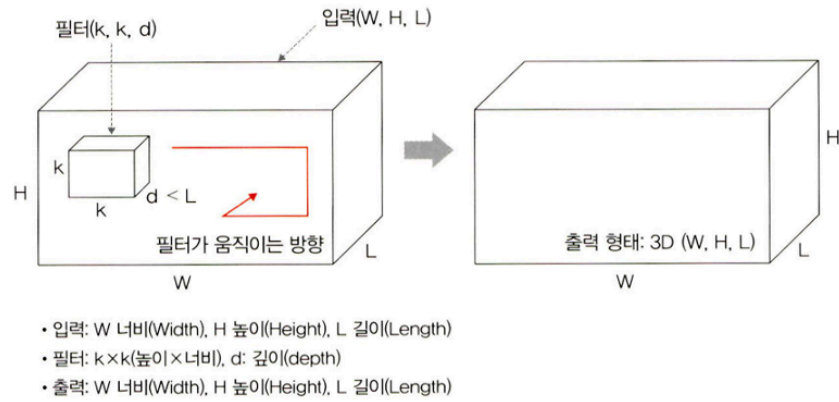


필터가 방향 2개로 움직이는 형태

입력 (W, H)과 필터 (k, k)에 대한 출력은 (W, H)가 됨

출력 형태는 2D 행렬

- 3D 합성곱



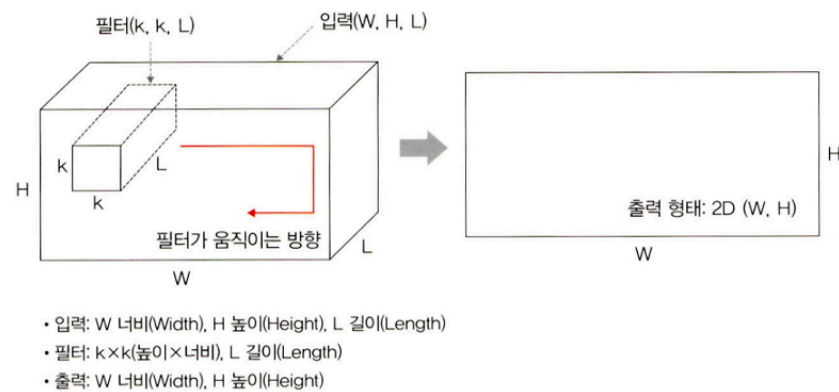
필터가 움직이는 방향이 3개

입력 (W, H, L)에 대해 필터 (k, k, d)를 적용하면 출력으로 (W, H, L)을 가짐

출력 형태는 3D 형태

$d < L$ 을 유지해야 함

- 3D 입력을 갖는 2D 합성곱



입력이 3D 형태임에도 출력 형태가 3D가 아닌 2D 행렬을 취하는 것

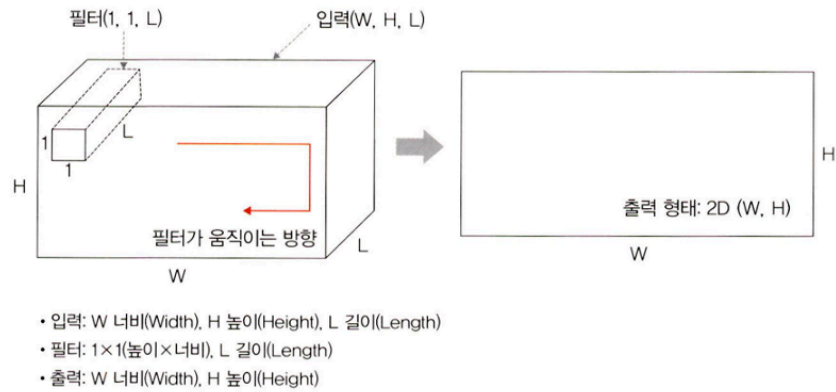
필터에 대한 길이 L이 입력 채널의 길이 L과 같아야 하기 때문에 합성곱 형태가 만들어짐

입력 (W, H, L)에 필터 (k, k, L)를 적용하면 출력은 (W, H)가 됨

필터는 두 방향으로 움직이며 출력 형태는 2D 행렬이 됨

대표적으로는 LeNet-5, VGG가 있음

- 1x1 합성곱



3D 형태로 입력됨

입력 (W, H, L)에 필터 (1, 1, L)을 적용하면 출력은 (W, H)가 됨

채널 수를 조정해서 연산량이 감소되는 효과가 있음

대표적으로는 GoogLeNet이 있음

5.2 합성곱 신경망 맛보기

fashion_minist 데이터셋을 사용하여 합성곱 신경망을 직접 구현

```
#데이터를 데이터로더에 전달
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size = 100)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size = 100)
```

torch.utils.data.DataLoader()를 사용하여 원하는 크기의 배치 단위로 데이터를 불러오거나, 순서가 무작위로 섞이도록 할 수 있음

(데이터를 불러올 데이터셋 지정, 데이터를 배치로 묶어줌)

코드에서는 batch_size = 100으로 지정했기 때문에 100개 단위로 데이터를 묶어서 불러옴

```
#분류에 사용될 클래스 정의
labels_map = {0 : 'T-shirt', 1 : 'Trouser', 2 : 'Pullover',
3 : 'Dress', 4 : 'Coat',
5 : 'Sandal', 6 : 'Shirt', 7 : 'Sneaker', 8 : 'Bag', 9 : 'Ankle Boot'}
```

```
#20개의 이미지 데이터를 시각적으로 표현
```

```

fig = plt.figure(figsize = (8, 8));
columns = 4;
rows = 5;
for i in range(1, columns * rows + 1):
    #이산형 분포를 갖는 데이터에서 무작위 표본을 추출
    #0~(train_dataset의 길이) 값을 갖는 분포에서 랜덤한 숫자 한 개
    #를 생성
    img_xy = np.random.randint(len(train_dataset));
    #train_dataset을 이용한 3차원 배열 생성
    img = train_dataset[img_xy][0][0,:,:]
    fig.add_subplot(rows, columns, i)
    plt.title(labels_map[train_dataset[img_xy][1]])
    plt.axis('off')
    plt.imshow(img, cmap = 'gray')
plt.show()

```

np.random은 무작위로 데이터를 생성할 때 사용

예시)

np.random.randint(10) → 0~10의 임의의 숫자 출력

np.random.randint(1, 10) → 1~9의 임의의 숫자 출력

np.random.rand(8) → 0~1 사이의 정규표준분포 난수를 행렬로 (1x8) 출력

np.random.rand(4, 2) → 0~1 사이의 정규표준분포 난수를 행렬로 (4x2) 출력

np.random.randn(8) → 평균이 0이고 표준편차가 1인 가우시안 정규분포 난수를 행렬로 (1x8) 출력

img = train_dataset[img_xy][0][0,:,:]는 인덱스에 해당하는 요소 값을 가져오겠다는 의미

비교를 위한 심층 신경망 생성 (ConvNet이 적용되지 않은 네트워크 생성)

```

#심층 신경망 모델 생성
class FashionDNN(nn.Module): #class 형태의 모델은 항상 torch.
nn.Module을 상속받음
    def __init__(self): #객체가 갖는 속성 값을 초기화하는 역할, 객
체가 생성될 때 자동으로 호출
        super(FashionDNN, self).__init__() #FashionDNN이라는 부
모 클래스를 상속받음
        #in_features = 입력의 크기, out_features = 출력의 크기

```

```

#nn : 딥러닝 모델 구성에 필요한 모듈이 모여 있는 패키지, Linear
#는 단순 선형 회귀 모델을 만들 때 사용
self.fc1 = nn.Linear(in_features = 784, out_features =
256)
self.drop = nn.Dropout(0.25) #0.25만큼 비율로 텐서의 값이
0이 됨, 0이 되지 않는 값들은 커짐
self.fc2 = nn.Linear(in_features = 256, out_features =
128)
self.fc3 = nn.Linear(in_features = 128, out_features =
10)

def forward(self, input_data): #순전파 학습을 진행, 객체를 데
이터와 함께 호출하면 자동으로 실행됨
#view는 넘파이의 reshape과 같은 역할로 텐서의 크기를 변경
#input_data를 (-1, 784)의 크기로 변경, -1의 값은 직접 계산해
야함
out = input_data.view(-1, 784)
#활성화 함수는 __init__, forward 둘 다에서 정의 가능
out = F.relu(self.fc1(out))
out = self.drop(out)
out = F.relu(self.fc2(out))
out = self.fc3(out)
return out

```

```

#심층 신경망에서 필요한 파라미터 정의
learning_rate = 0.001;
model = FashionDNN();
model.to(device)

criterion = nn.CrossEntropyLoss();
#optimizer을 위한 경사 하강법은 Adam 사용, 학습률은 0.001 사용
optimizer = torch.optim.Adam(model.parameters(), lr = learn
ing_rate);
print(model)

```

```

#심층 신경망을 이용한 모델 학습
num_epochs = 5

```

```

count = 0

loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader:
        #모델이 데이터를 처리하기 위해 모델과 데이터가 동일한 장치에 있어야 함
        images, labels = images.to(device), labels.to(device)

        #Autograd로 자동 미분된 값을 저장하기 위해 tape 사용
        #순전파에서 tape는 수행하는 모든 연산을 저장, 역전파에서 저장된 값들을 꺼내서 사용
        #Autograd는 Variable을 사용해서 역전파를 위한 미분 값을 자동으로 계산
        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels)

        outputs = model(train)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1

    if not (count % 50): #count를 50으로 나누었을 때 나머지가 0
        이 아니면 실행
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)

            labels_list.append(labels)

```

```

test = Variable(images.view(100, 1, 28, 28))
outputs = model(test)
predictions = torch.max(outputs, 1)[1].to(device)
predictions_list.append(predictions)
correct += (predictions == labels).sum()
total += len(labels)

accuracy = correct * 100 / total
loss_list.append(loss.data)
iteration_list.append(count)
accuracy_list.append(accuracy)

if not (count % 500):
    print("Iteration: {}, Loss: {}, Accuracy: {}%".format
(count, loss.data, accuracy))

```

▼ Variable의 역할?

variable = 과거 파이토치에서 자동 미분을 위해 텐서를 감싸던 객체

images = Variable(images)

: 이 images를 계산에 사용하고, 필요하면 이 계산 과정을 추적해서 나중에 역전파할 수 있게 함

variable은 학습에 사용할 데이터

Variable(images)는 이 images 데이터를 딥러닝 계산에 사용할거라고 감싸주는 역할
예전 파이토치에서는 데이터를 그냥 사용하면 미분을 추적하는 기능이 부족해서

Variable를 씌웠음

Variable이 계산 과정을 추적해서 나중에 역전파를 할 수 있도록 도와주는 역할을 함

최신 파이토치에서는 tensor에 기능이 통합되어 거의 사용하지 않음

```

Iteration: 500, Loss: 0.5912195444107056, Accuracy: 83.7300033569336%
Iteration: 1000, Loss: 0.42913830280303955, Accuracy: 85.06999969482422%
Iteration: 1500, Loss: 0.33972546458244324, Accuracy: 83.86000061035156%
Iteration: 2000, Loss: 0.42390331625938416, Accuracy: 84.97000122070312%
Iteration: 2500, Loss: 0.28261277079582214, Accuracy: 86.0199966430664%
Iteration: 3000, Loss: 0.2839193046092987, Accuracy: 85.86000061035156%

```

심층 신경망 모델 학습 결과

합성곱 신경망 생성

#합성곱 네트워크 생성

```
class FashionCNN(nn.Module):
```

```
    def __init__(self):
```

```
        super(FashionCNN, self).__init__()
```

#nn.Sequential()은 계층을 차례로 쌓을 수 있도록 $wx + b$ 와 같은 수식과 활성화 함수를 연결해주는 역할

#여러 개의 계층을 하나의 컨테이너에 구현하는 방법

```
    self.layer1 = nn.Sequential(
```

#(입력 채널의 수, 출력 채널의 수, 필터 크기, 출력 크기를 조정하기 위해 주위에 0으로 채움)

```
        nn.Conv2d(in_channels = 1, out_channels = 32, kernel_size = 3, padding = 1), #패딩 값이 클수록 출력 크기도 커짐
```

```
        nn.BatchNorm2d(32), #각 배치 단위별로 데이터가 다양한 분포를 가져도 평균과 분산을 이용하여 정규화함
```

```
        nn.ReLU(),
```

#이미지 크기를 축소시키는 용도

#($m \times n$ 행렬로 구성된 가중치, 이동할 간격)

```
        nn.MaxPool2d(kernel_size = 2, stride = 2) #스트라이드 값이 커지면 출력 크기는 작아짐
```

```
    )
```

```
    self.layer2 = nn.Sequential(
```

```
        nn.Conv2d(in_channels = 32, out_channels = 64, kernel_size = 3),
```

```
        nn.BatchNorm2d(64),
```

```
        nn.ReLU(),
```

```
        nn.MaxPool2d(2)
```

```
    )
```

#클래스를 분류하기 위해 이미지 형태의 데이터를 배열 형태로 변환하여 작업

#Conv2d에서 사용하는 하이퍼파라미터 값들에 따라 출력 크기가 달라짐

```
    self.fc1 = nn.Linear(in_features = 64*6*6, out_features = 600)
```

```
    self.drop = nn.Dropout2d(0.25)
```

```
    self.fc2 = nn.Linear(in_features = 600, out_features = 120)
```

```
    self.fc3 = nn.Linear(in_features = 120, out_features = 10) #마지막 계층의 out_features는 클래스수 개수
```

```

def forward(self, x):
    out = self.layer1(x)
    out = self.layer2(out)
    #합성곱층에서 완전 연결층으로 변경되기 때문에 데이터의 형태를 1
    #차원으로 바꿔줌
    out = out.view(out.size(0), -1) #out.size(0)은 100을 의
    #미 -> (100, ?) 크기의 텐서로 변경
    out = self.fc1(out)
    out = self.drop(out)
    out = self.fc2(out)
    out = self.fc3(out)
    return out

```

```

#합성곱 네트워크를 위한 파라미터 정의
learning_rate = 0.001;
model = FashionCNN();
model.to(device)

criterion = nn.CrossEntropyLoss();
optimizer = torch.optim.Adam(model.parameters(), lr = learn
ing_rate);
print(model)

```

```

#모델 학습 및 성능 평가
num_epochs = 5
count = 0
loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

```

```

train = Variable(images.view(100, 1, 28, 28))
labels = Variable(labels)

outputs = model(train)
loss = criterion(outputs, labels)
optimizer.zero_grad()
loss.backward()
optimizer.step()
count += 1

if not (count % 50):
    total = 0
    correct = 0
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)

        labels_list.append(labels)
        test = Variable(images.view(100, 1, 28, 28))
        outputs = model(test)
        predictions = torch.max(outputs, 1)[1].to(device)
        predictions_list.append(predictions)
        correct += (predictions == labels).sum()
        total += len(labels)

    accuracy = correct * 100 / total
    loss_list.append(loss.data)
    iteration_list.append(count)
    accuracy_list.append(accuracy)

if not (count % 500):
    print("Iteration: {}, Loss: {}, Accuracy: {}".format(
count, loss.data, accuracy))

```

```
Iteration: 500, Loss: 0.45545855164527893, Accuracy: 87.88999938964844%  
Iteration: 1000, Loss: 0.31720197200775146, Accuracy: 88.91000366210938%  
Iteration: 1500, Loss: 0.3124687075614929, Accuracy: 88.56999969482422%  
Iteration: 2000, Loss: 0.21635320782661438, Accuracy: 89.58999633789062%  
Iteration: 2500, Loss: 0.10785746574401855, Accuracy: 89.62999725341797%  
Iteration: 3000, Loss: 0.17871271073818207, Accuracy: 90.58000183105469%
```

심층 신경망 모델 학습 결과

심층 신경망과 비교하여 정확도가 약간 높음

정확도가 조금 밖에 차이가 안나서 간편한 심층 신경망만 사용해도 될 것 같지만 실제로 이
미지 데이터가 많아지면 단순 심층 신경망으로는 정확한 특성 추출 및 분류가 불가능함
때문에 합성곱 신경망을 생성할 수 있도록 학습해야함